Foundations of Computing

A demo course for the Defter reader



Six short chapters

1 Logic and Propositions

2 Sets and Functions

3 Algorithms and Complexity

4 Data Structures

5 Graphs

6 Finite Automata

This document was generated by a script for testing. All text and figures were written for it; it contains no copyrighted material and may be shared freely.

About this document

Defter shows notes from Google Drive as books in a 3D study room. To try the reader without a Google account, the sample library opens this document and a set of landscape slides. Both are made by scripts/demo-pdf-uret.mjs, which writes the PDF files directly.

The document is long on purpose. A real lecture script has dozens of pages, and the reader keeps only the visible pages and two on each side as drawn canvases. Scrolling through more than sixty pages shows that memory stays low while pages appear quickly.

The text explains a few basic ideas of computer science in plain language. Some sections repeat ideas from earlier pages; that is fine for a test document. Figures use simple lines, boxes and circles, so they stay sharp at every zoom level.

Try the following: switch between book mode and PDF mode, open the table of contents, drag the page slider, zoom into a figure, and close the reader in the middle of a chapter. When you open the topic again, the ribbon brings you back to the same page.

Contents

1 Logic and Propositions 4

2 Sets and Functions 13

3 Algorithms and Complexity 22

4 Data Structures 31

5 Graphs 40

6 Finite Automata 49

Exercises 58

Glossary 62

Chapter 1

Logic and Propositions

A proposition is a sentence that is either true or false, but not both. 'It is raining in Bremen' is a proposition; 'Close the window' is not, because it has no truth value. Logic gives us a small set of tools to combine propositions and to decide, without guessing, whether a combined statement holds.

p	q	p and q	p or q	if p then q
T	T	T	T	T
T	F	F	T	F
F	T	F	T	T
F	F	F	F	T

1.1 Propositions

A proposition is a sentence that is either true or false, but not both. 'It is raining in Bremen' is a proposition; 'Close the window' is not, because it has no truth value. Logic gives us a small set of tools to combine propositions and to decide, without guessing, whether a combined statement holds.

The basic connectives are negation (not), conjunction (and), disjunction (or) and implication (if ... then). Each one is defined completely by a truth table: for every combination of truth values of its parts, the table tells us the value of the whole. Two formulas are equivalent when their tables are identical.

p	q	p and q	p or q	if p then q
T	T	T	T	T
T	F	F	T	F
F	T	F	T	T
F	F	F	F	T

Figure 1.1 Truth table of the basic connectives

Implication is the connective that surprises most beginners. 'If p then q' is false in exactly one case: p is true and q is false. When p is false, the implication is true no matter what q says. This is why a promise is only broken when its condition happened and the result did not.

Laws such as De Morgan's rules let us rewrite formulas: not (p and q) is the same as (not p) or (not q). Rewriting is useful in programming, too. A long condition in an if statement can often be simplified into a shorter one that is easier to read and to test.

A formula that is true in every row of its table is a tautology; one that is false in every row is a contradiction. Checking a table with n variables needs 2^n rows, so for larger problems we use proof rules or clever algorithms instead of writing every row by hand.

1.2 Connectives

The basic connectives are negation (not), conjunction (and), disjunction (or) and implication (if ... then). Each one is defined completely by a truth table: for every combination of truth values of its parts, the table tells us the value of the whole. Two formulas are equivalent when their tables are identical.

Implication is the connective that surprises most beginners. 'If p then q' is false in exactly one case: p is true and q is false. When p is false, the implication is true no matter what q says. This is why a promise is only broken when its condition happened and the result did not.

> **Key idea**
> **A truth table is a complete definition: nothing about a connective is left open.**

Laws such as De Morgan's rules let us rewrite formulas: not (p and q) is the same as (not p) or (not q). Rewriting is useful in programming, too. A long condition in an if statement can often be simplified into a shorter one that is easier to read and to test.

A formula that is true in every row of its table is a tautology; one that is false in every row is a contradiction. Checking a table with n variables needs 2^n rows, so for larger problems we use proof rules or clever algorithms instead of writing every row by hand.

A proposition is a sentence that is either true or false, but not both. 'It is raining in Bremen' is a proposition; 'Close the window' is not, because it has no truth value. Logic gives us a small set of tools to combine propositions and to decide, without guessing, whether a combined statement holds.

1.3 Truth tables

Implication is the connective that surprises most beginners. 'If p then q' is false in exactly one case: p is true and q is false. When p is false, the implication is true no matter what q says. This is why a promise is only broken when its condition happened and the result did not.

Laws such as De Morgan's rules let us rewrite formulas: not (p and q) is the same as (not p) or (not q). Rewriting is useful in programming, too. A long condition in an if statement can often be simplified into a shorter one that is easier to read and to test.

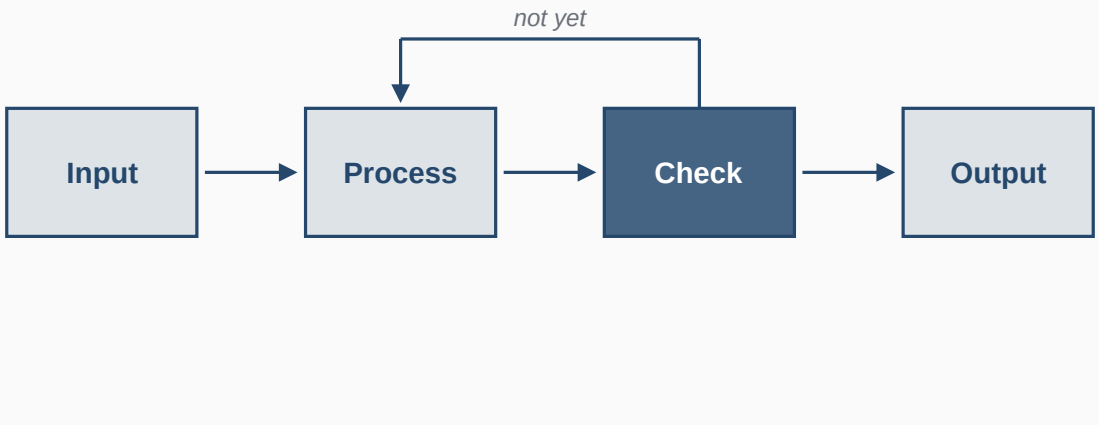


Figure 1.2 A process with a check that repeats a step

A formula that is true in every row of its table is a tautology; one that is false in every row is a contradiction. Checking a table with n variables needs 2^n rows, so for larger problems we use proof rules or clever algorithms instead of writing every row by hand.

A proposition is a sentence that is either true or false, but not both. 'It is raining in Bremen' is a proposition; 'Close the window' is not, because it has no truth value. Logic gives us a small set of tools to combine propositions and to decide, without guessing, whether a combined statement holds.

The basic connectives are negation (not), conjunction (and), disjunction (or) and implication (if ... then). Each one is defined completely by a truth table: for every combination of truth values of its parts, the table tells us the value of the whole. Two formulas are equivalent when their tables are identical.

1.4 Implication

Laws such as De Morgan's rules let us rewrite formulas: not (p and q) is the same as (not p) or (not q). Rewriting is useful in programming, too. A long condition in an if statement can often be simplified into a shorter one that is easier to read and to test.

A formula that is true in every row of its table is a tautology; one that is false in every row is a contradiction. Checking a table with n variables needs 2^n rows, so for larger problems we use proof rules or clever algorithms instead of writing every row by hand.

> **Key idea**
> **A truth table is a complete definition: nothing about a connective is left open.**

A proposition is a sentence that is either true or false, but not both. 'It is raining in Bremen' is a proposition; 'Close the window' is not, because it has no truth value. Logic gives us a small set of tools to combine propositions and to decide, without guessing, whether a combined statement holds.

The basic connectives are negation (not), conjunction (and), disjunction (or) and implication (if ... then). Each one is defined completely by a truth table: for every combination of truth values of its parts, the table tells us the value of the whole. Two formulas are equivalent when their tables are identical.

Implication is the connective that surprises most beginners. 'If p then q' is false in exactly one case: p is true and q is false. When p is false, the implication is true no matter what q says. This is why a promise is only broken when its condition happened and the result did not.

1.5 Equivalence

A formula that is true in every row of its table is a tautology; one that is false in every row is a contradiction. Checking a table with n variables needs 2^n rows, so for larger problems we use proof rules or clever algorithms instead of writing every row by hand.

A proposition is a sentence that is either true or false, but not both. 'It is raining in Bremen' is a proposition; 'Close the window' is not, because it has no truth value. Logic gives us a small set of tools to combine propositions and to decide, without guessing, whether a combined statement holds.

p	q	p and q	p or q	if p then q
T	T	T	T	T
T	F	F	T	F
F	T	F	T	T
F	F	F	F	T

Figure 1.3 Truth table of the basic connectives

The basic connectives are negation (not), conjunction (and), disjunction (or) and implication (if ... then). Each one is defined completely by a truth table: for every combination of truth values of its parts, the table tells us the value of the whole. Two formulas are equivalent when their tables are identical.

Implication is the connective that surprises most beginners. 'If p then q' is false in exactly one case: p is true and q is false. When p is false, the implication is true no matter what q says. This is why a promise is only broken when its condition happened and the result did not.

Laws such as De Morgan's rules let us rewrite formulas: not (p and q) is the same as (not p) or (not q). Rewriting is useful in programming, too. A long condition in an if statement can often be simplified into a shorter one that is easier to read and to test.

1.6 De Morgan's laws

A proposition is a sentence that is either true or false, but not both. 'It is raining in Bremen' is a proposition; 'Close the window' is not, because it has no truth value. Logic gives us a small set of tools to combine propositions and to decide, without guessing, whether a combined statement holds.

The basic connectives are negation (not), conjunction (and), disjunction (or) and implication (if ... then). Each one is defined completely by a truth table: for every combination of truth values of its parts, the table tells us the value of the whole. Two formulas are equivalent when their tables are identical.

> **Key idea**
> **A truth table is a complete definition: nothing about a connective is left open.**

Implication is the connective that surprises most beginners. 'If p then q' is false in exactly one case: p is true and q is false. When p is false, the implication is true no matter what q says. This is why a promise is only broken when its condition happened and the result did not.

Laws such as De Morgan's rules let us rewrite formulas: not (p and q) is the same as (not p) or (not q). Rewriting is useful in programming, too. A long condition in an if statement can often be simplified into a shorter one that is easier to read and to test.

A formula that is true in every row of its table is a tautology; one that is false in every row is a contradiction. Checking a table with n variables needs 2^n rows, so for larger problems we use proof rules or clever algorithms instead of writing every row by hand.

1.7 Tautologies

The basic connectives are negation (not), conjunction (and), disjunction (or) and implication (if ... then). Each one is defined completely by a truth table: for every combination of truth values of its parts, the table tells us the value of the whole. Two formulas are equivalent when their tables are identical.

Implication is the connective that surprises most beginners. 'If p then q' is false in exactly one case: p is true and q is false. When p is false, the implication is true no matter what q says. This is why a promise is only broken when its condition happened and the result did not.

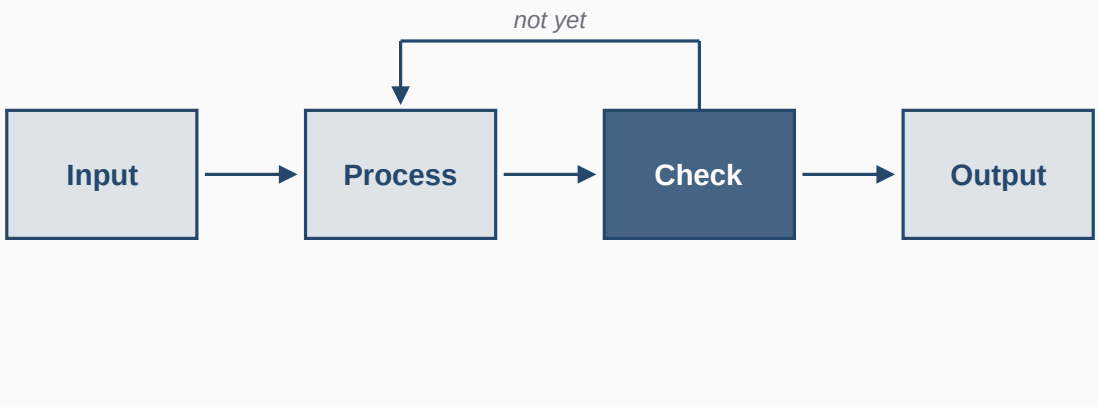


Figure 1.4 A process with a check that repeats a step

Laws such as De Morgan's rules let us rewrite formulas: not (p and q) is the same as (not p) or (not q). Rewriting is useful in programming, too. A long condition in an if statement can often be simplified into a shorter one that is easier to read and to test.

A formula that is true in every row of its table is a tautology; one that is false in every row is a contradiction. Checking a table with n variables needs 2^n rows, so for larger problems we use proof rules or clever algorithms instead of writing every row by hand.

A proposition is a sentence that is either true or false, but not both. 'It is raining in Bremen' is a proposition; 'Close the window' is not, because it has no truth value. Logic gives us a small set of tools to combine propositions and to decide, without guessing, whether a combined statement holds.

1.8 Practice

Implication is the connective that surprises most beginners. 'If p then q' is false in exactly one case: p is true and q is false. When p is false, the implication is true no matter what q says. This is why a promise is only broken when its condition happened and the result did not.

Laws such as De Morgan's rules let us rewrite formulas: not (p and q) is the same as (not p) or (not q). Rewriting is useful in programming, too. A long condition in an if statement can often be simplified into a shorter one that is easier to read and to test.

> **Key idea**
> **A truth table is a complete definition: nothing about a connective is left open.**

A formula that is true in every row of its table is a tautology; one that is false in every row is a contradiction. Checking a table with n variables needs 2^n rows, so for larger problems we use proof rules or clever algorithms instead of writing every row by hand.

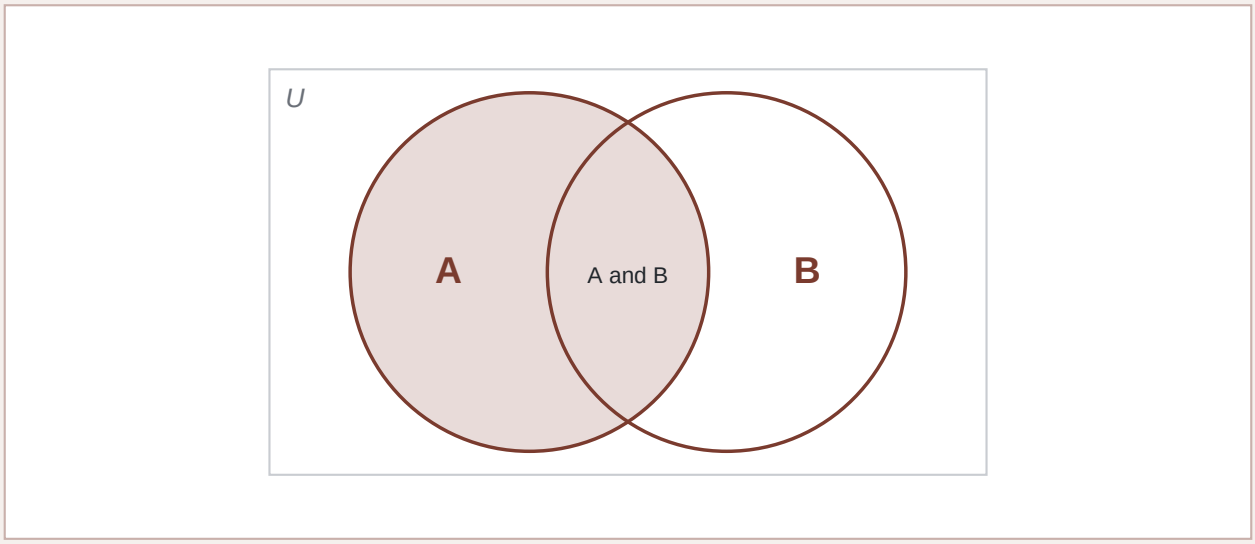
A proposition is a sentence that is either true or false, but not both. 'It is raining in Bremen' is a proposition; 'Close the window' is not, because it has no truth value. Logic gives us a small set of tools to combine propositions and to decide, without guessing, whether a combined statement holds.

The basic connectives are negation (not), conjunction (and), disjunction (or) and implication (if ... then). Each one is defined completely by a truth table: for every combination of truth values of its parts, the table tells us the value of the whole. Two formulas are equivalent when their tables are identical.

Chapter 2

Sets and Functions

A set is a collection of distinct objects, called its elements. The order does not matter and nothing appears twice: {1, 2, 3} and {3, 1, 2} are the same set. We write x in A when x is an element of A, and we describe large sets with a rule instead of a list.



2.1 Sets and elements

A set is a collection of distinct objects, called its elements. The order does not matter and nothing appears twice: {1, 2, 3} and {3, 1, 2} are the same set. We write x in A when x is an element of A, and we describe large sets with a rule instead of a list.

Union, intersection and difference build new sets from old ones. The union of A and B contains everything that is in at least one of them; the intersection contains only what they share. A Venn diagram is a quick way to picture these operations, although it is not a proof.

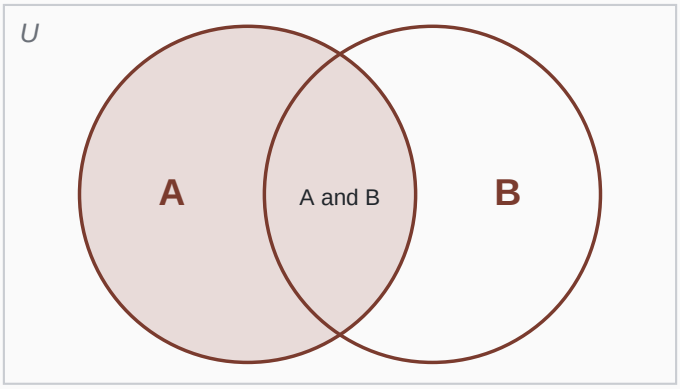


Figure 2.1 Two sets inside a universe U

A function f from A to B assigns exactly one element of B to every element of A. A is the domain and B is the codomain. A function is injective if different inputs always give different outputs, and surjective if every element of B is reached by some input.

A function that is both injective and surjective is a bijection. Bijections are exactly the functions that can be reversed. They also let us compare the sizes of sets: two finite sets have the same number of elements precisely when there is a bijection between them.

Relations generalise functions. A relation between A and B is any set of pairs (a, b). Ordering relations such as 'less than or equal' and equivalence relations such as 'has the same remainder modulo 3' appear again and again in computer science.

2.2 Describing sets

Union, intersection and difference build new sets from old ones. The union of A and B contains everything that is in at least one of them; the intersection contains only what they share. A Venn diagram is a quick way to picture these operations, although it is not a proof.

A function f from A to B assigns exactly one element of B to every element of A. A is the domain and B is the codomain. A function is injective if different inputs always give different outputs, and surjective if every element of B is reached by some input.

> **Key idea**
> **A function gives every input exactly one output; a bijection can also be reversed.**

A function that is both injective and surjective is a bijection. Bijections are exactly the functions that can be reversed. They also let us compare the sizes of sets: two finite sets have the same number of elements precisely when there is a bijection between them.

Relations generalise functions. A relation between A and B is any set of pairs (a, b). Ordering relations such as 'less than or equal' and equivalence relations such as 'has the same remainder modulo 3' appear again and again in computer science.

A set is a collection of distinct objects, called its elements. The order does not matter and nothing appears twice: {1, 2, 3} and {3, 1, 2} are the same set. We write x in A when x is an element of A, and we describe large sets with a rule instead of a list.

2.3 Set operations

A function f from A to B assigns exactly one element of B to every element of A. A is the domain and B is the codomain. A function is injective if different inputs always give different outputs, and surjective if every element of B is reached by some input.

A function that is both injective and surjective is a bijection. Bijections are exactly the functions that can be reversed. They also let us compare the sizes of sets: two finite sets have the same number of elements precisely when there is a bijection between them.

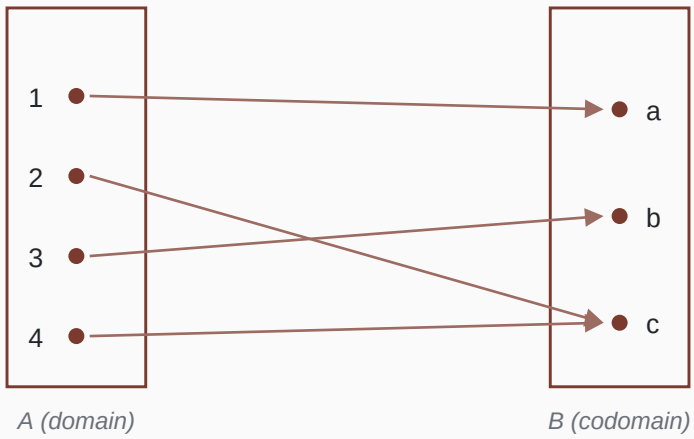


Figure 2.2 A function from A to B

Relations generalise functions. A relation between A and B is any set of pairs (a, b). Ordering relations such as 'less than or equal' and equivalence relations such as 'has the same remainder modulo 3' appear again and again in computer science.

A set is a collection of distinct objects, called its elements. The order does not matter and nothing appears twice: {1, 2, 3} and {3, 1, 2} are the same set. We write x in A when x is an element of A, and we describe large sets with a rule instead of a list.

Union, intersection and difference build new sets from old ones. The union of A and B contains everything that is in at least one of them; the intersection contains only what they share. A Venn diagram is a quick way to picture these operations, although it is not a proof.

2.4 Venn diagrams

A function that is both injective and surjective is a bijection. Bijections are exactly the functions that can be reversed. They also let us compare the sizes of sets: two finite sets have the same number of elements precisely when there is a bijection between them.

Relations generalise functions. A relation between A and B is any set of pairs (a, b). Ordering relations such as 'less than or equal' and equivalence relations such as 'has the same remainder modulo 3' appear again and again in computer science.

> **Key idea**
> **A function gives every input exactly one output; a bijection can also be reversed.**

A set is a collection of distinct objects, called its elements. The order does not matter and nothing appears twice: {1, 2, 3} and {3, 1, 2} are the same set. We write x in A when x is an element of A, and we describe large sets with a rule instead of a list.

Union, intersection and difference build new sets from old ones. The union of A and B contains everything that is in at least one of them; the intersection contains only what they share. A Venn diagram is a quick way to picture these operations, although it is not a proof.

A function f from A to B assigns exactly one element of B to every element of A. A is the domain and B is the codomain. A function is injective if different inputs always give different outputs, and surjective if every element of B is reached by some input.

2.5 Functions

Relations generalise functions. A relation between A and B is any set of pairs (a, b). Ordering relations such as 'less than or equal' and equivalence relations such as 'has the same remainder modulo 3' appear again and again in computer science.

A set is a collection of distinct objects, called its elements. The order does not matter and nothing appears twice: {1, 2, 3} and {3, 1, 2} are the same set. We write x in A when x is an element of A, and we describe large sets with a rule instead of a list.

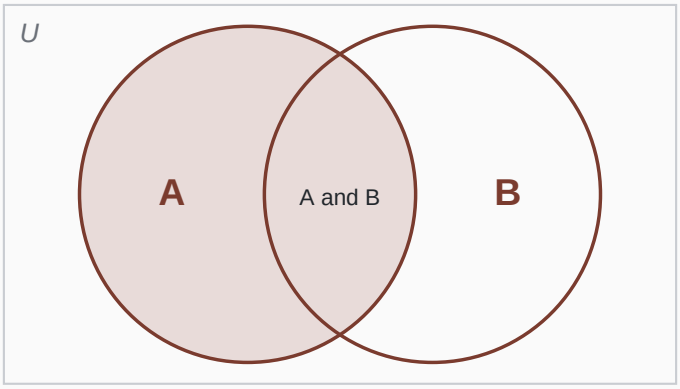


Figure 2.3 Two sets inside a universe U

Union, intersection and difference build new sets from old ones. The union of A and B contains everything that is in at least one of them; the intersection contains only what they share. A Venn diagram is a quick way to picture these operations, although it is not a proof.

A function f from A to B assigns exactly one element of B to every element of A. A is the domain and B is the codomain. A function is injective if different inputs always give different outputs, and surjective if every element of B is reached by some input.

A function that is both injective and surjective is a bijection. Bijections are exactly the functions that can be reversed. They also let us compare the sizes of sets: two finite sets have the same number of elements precisely when there is a bijection between them.

2.6 Injective and surjective

A set is a collection of distinct objects, called its elements. The order does not matter and nothing appears twice: {1, 2, 3} and {3, 1, 2} are the same set. We write x in A when x is an element of A, and we describe large sets with a rule instead of a list.

Union, intersection and difference build new sets from old ones. The union of A and B contains everything that is in at least one of them; the intersection contains only what they share. A Venn diagram is a quick way to picture these operations, although it is not a proof.

> **Key idea**
> **A function gives every input exactly one output; a bijection can also be reversed.**

A function f from A to B assigns exactly one element of B to every element of A. A is the domain and B is the codomain. A function is injective if different inputs always give different outputs, and surjective if every element of B is reached by some input.

A function that is both injective and surjective is a bijection. Bijections are exactly the functions that can be reversed. They also let us compare the sizes of sets: two finite sets have the same number of elements precisely when there is a bijection between them.

Relations generalise functions. A relation between A and B is any set of pairs (a, b). Ordering relations such as 'less than or equal' and equivalence relations such as 'has the same remainder modulo 3' appear again and again in computer science.

2.7 Bijections

Union, intersection and difference build new sets from old ones. The union of A and B contains everything that is in at least one of them; the intersection contains only what they share. A Venn diagram is a quick way to picture these operations, although it is not a proof.

A function f from A to B assigns exactly one element of B to every element of A. A is the domain and B is the codomain. A function is injective if different inputs always give different outputs, and surjective if every element of B is reached by some input.

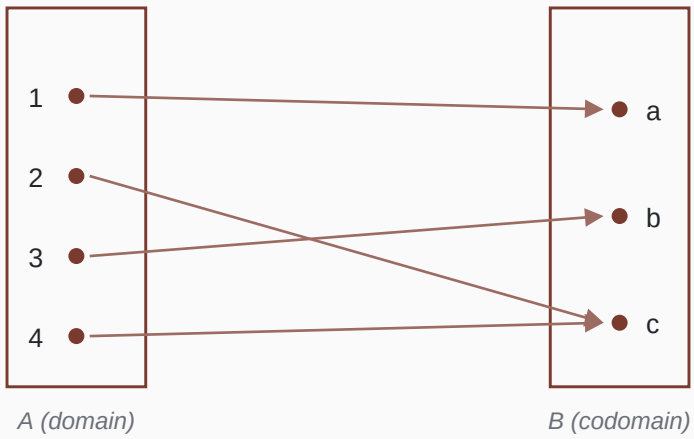


Figure 2.4 A function from A to B

A function that is both injective and surjective is a bijection. Bijections are exactly the functions that can be reversed. They also let us compare the sizes of sets: two finite sets have the same number of elements precisely when there is a bijection between them.

Relations generalise functions. A relation between A and B is any set of pairs (a, b). Ordering relations such as 'less than or equal' and equivalence relations such as 'has the same remainder modulo 3' appear again and again in computer science.

A set is a collection of distinct objects, called its elements. The order does not matter and nothing appears twice: {1, 2, 3} and {3, 1, 2} are the same set. We write x in A when x is an element of A, and we describe large sets with a rule instead of a list.

2.8 Relations

A function f from A to B assigns exactly one element of B to every element of A. A is the domain and B is the codomain. A function is injective if different inputs always give different outputs, and surjective if every element of B is reached by some input.

A function that is both injective and surjective is a bijection. Bijections are exactly the functions that can be reversed. They also let us compare the sizes of sets: two finite sets have the same number of elements precisely when there is a bijection between them.

> **Key idea**
> **A function gives every input exactly one output; a bijection can also be reversed.**

Relations generalise functions. A relation between A and B is any set of pairs (a, b). Ordering relations such as 'less than or equal' and equivalence relations such as 'has the same remainder modulo 3' appear again and again in computer science.

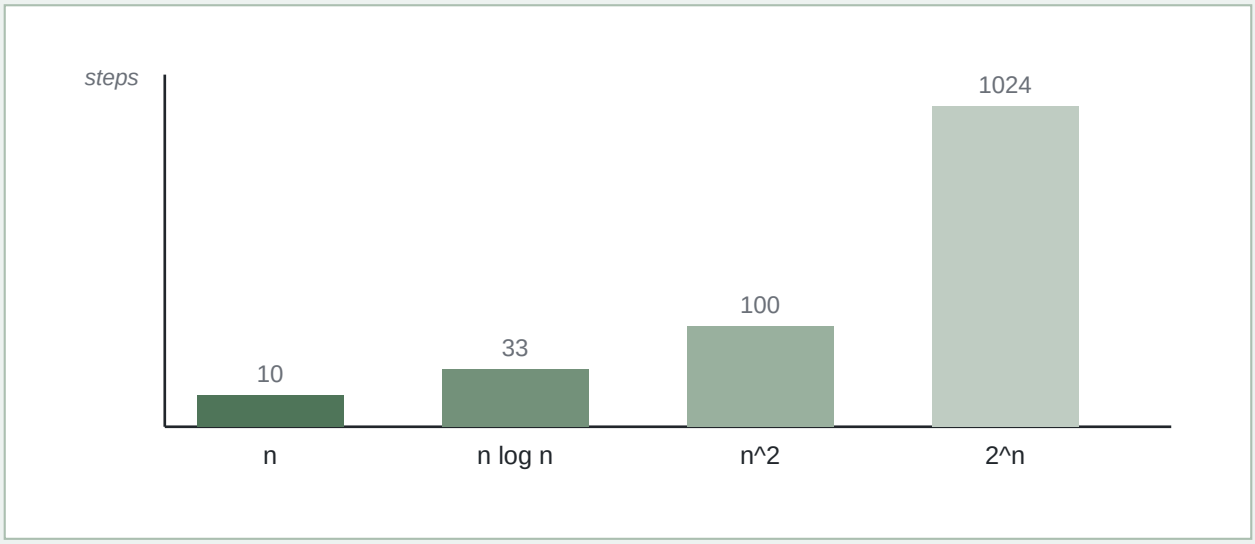
A set is a collection of distinct objects, called its elements. The order does not matter and nothing appears twice: {1, 2, 3} and {3, 1, 2} are the same set. We write x in A when x is an element of A, and we describe large sets with a rule instead of a list.

Union, intersection and difference build new sets from old ones. The union of A and B contains everything that is in at least one of them; the intersection contains only what they share. A Venn diagram is a quick way to picture these operations, although it is not a proof.

Chapter 3

Algorithms and Complexity

An algorithm is a finite list of precise steps that solves a problem for every allowed input. A recipe is a familiar example, but an algorithm must be exact enough that a machine can follow it without asking questions. Correctness and efficiency are the two properties we care about most.



3.1 What is an algorithm

An algorithm is a finite list of precise steps that solves a problem for every allowed input. A recipe is a familiar example, but an algorithm must be exact enough that a machine can follow it without asking questions. Correctness and efficiency are the two properties we care about most.

To compare algorithms we count basic steps as a function of the input size n. We are mostly interested in how this count grows when n becomes large, so we ignore constant factors and write the result in big-O notation, such as O(n), O(n log n) or O(n^2).

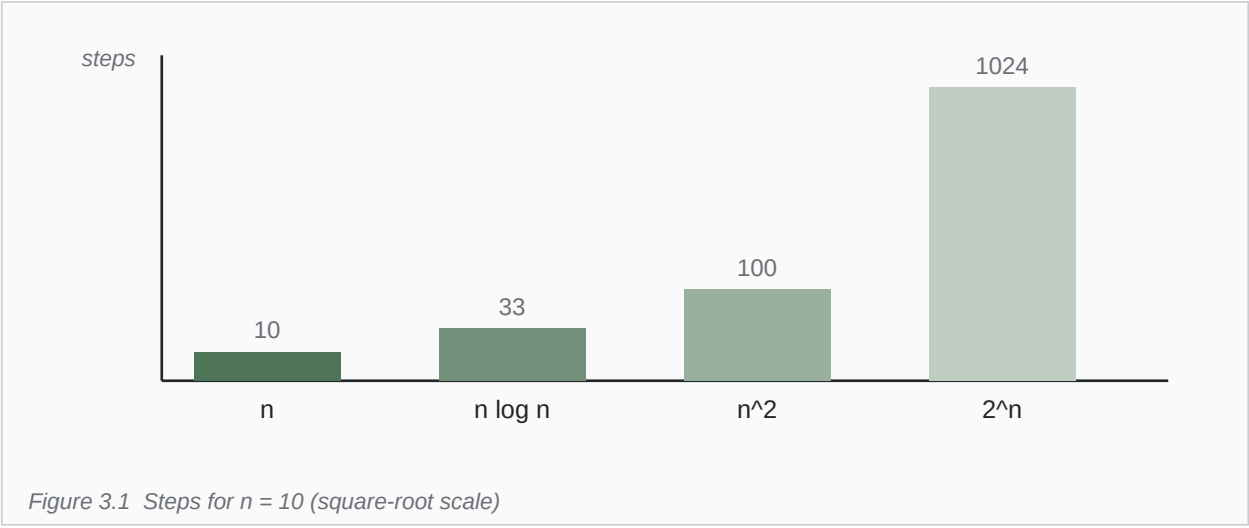


Figure 3.1 Steps for n = 10 (square-root scale)

Linear search looks at the elements one by one and needs up to n comparisons. Binary search works on a sorted list: it compares with the middle element and throws away half of the remaining list in every step. It therefore needs only about log2(n) comparisons.

Sorting is one of the most studied problems. Simple methods like insertion sort take O(n^2) steps in the worst case. Merge sort splits the list in halves, sorts each half and merges the results, which gives O(n log n) steps for every input.

Growth rates matter more than fast hardware. For a million elements, an O(n^2) method performs around a trillion steps, while an O(n log n) method needs about twenty million. A better algorithm can turn a computation of days into one of seconds.

3.2 Counting steps

To compare algorithms we count basic steps as a function of the input size n. We are mostly interested in how this count grows when n becomes large, so we ignore constant factors and write the result in big-O notation, such as O(n), O(n log n) or O(n^2).

Linear search looks at the elements one by one and needs up to n comparisons. Binary search works on a sorted list: it compares with the middle element and throws away half of the remaining list in every step. It therefore needs only about log2(n) comparisons.

> **Key idea**
> **For large inputs, the growth rate of an algorithm matters more than the speed of the machine.**

Sorting is one of the most studied problems. Simple methods like insertion sort take O(n^2) steps in the worst case. Merge sort splits the list in halves, sorts each half and merges the results, which gives O(n log n) steps for every input.

Growth rates matter more than fast hardware. For a million elements, an O(n^2) method performs around a trillion steps, while an O(n log n) method needs about twenty million. A better algorithm can turn a computation of days into one of seconds.

An algorithm is a finite list of precise steps that solves a problem for every allowed input. A recipe is a familiar example, but an algorithm must be exact enough that a machine can follow it without asking questions. Correctness and efficiency are the two properties we care about most.

3.3 Big-O notation

Linear search looks at the elements one by one and needs up to n comparisons. Binary search works on a sorted list: it compares with the middle element and throws away half of the remaining list in every step. It therefore needs only about log2(n) comparisons.

Sorting is one of the most studied problems. Simple methods like insertion sort take O(n^2) steps in the worst case. Merge sort splits the list in halves, sorts each half and merges the results, which gives O(n log n) steps for every input.

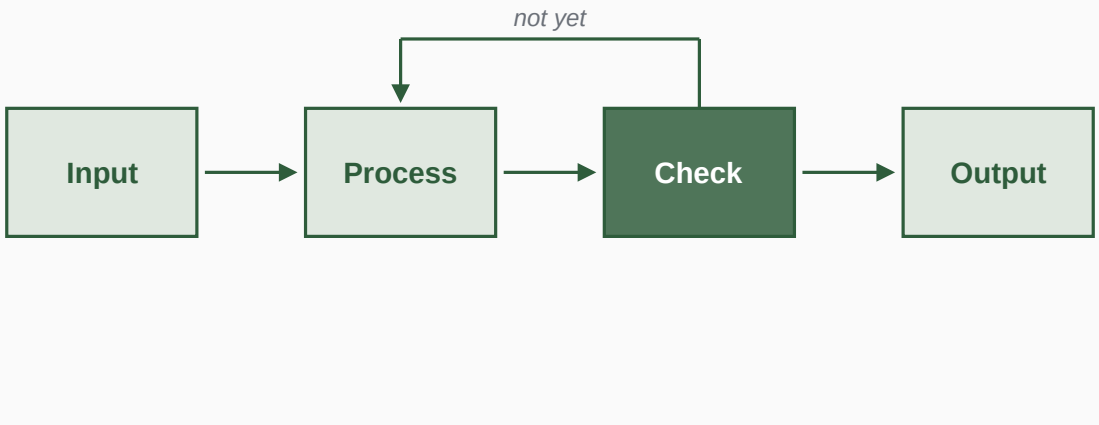


Figure 3.2 A process with a check that repeats a step

Growth rates matter more than fast hardware. For a million elements, an O(n^2) method performs around a trillion steps, while an O(n log n) method needs about twenty million. A better algorithm can turn a computation of days into one of seconds.

An algorithm is a finite list of precise steps that solves a problem for every allowed input. A recipe is a familiar example, but an algorithm must be exact enough that a machine can follow it without asking questions. Correctness and efficiency are the two properties we care about most.

To compare algorithms we count basic steps as a function of the input size n. We are mostly interested in how this count grows when n becomes large, so we ignore constant factors and write the result in big-O notation, such as O(n), O(n log n) or O(n^2).

3.4 Linear search

Sorting is one of the most studied problems. Simple methods like insertion sort take O(n^2) steps in the worst case. Merge sort splits the list in halves, sorts each half and merges the results, which gives O(n log n) steps for every input.

Growth rates matter more than fast hardware. For a million elements, an O(n^2) method performs around a trillion steps, while an O(n log n) method needs about twenty million. A better algorithm can turn a computation of days into one of seconds.

> **Key idea**
> **For large inputs, the growth rate of an algorithm matters more than the speed of the machine.**

An algorithm is a finite list of precise steps that solves a problem for every allowed input. A recipe is a familiar example, but an algorithm must be exact enough that a machine can follow it without asking questions. Correctness and efficiency are the two properties we care about most.

To compare algorithms we count basic steps as a function of the input size n. We are mostly interested in how this count grows when n becomes large, so we ignore constant factors and write the result in big-O notation, such as O(n), O(n log n) or O(n^2).

Linear search looks at the elements one by one and needs up to n comparisons. Binary search works on a sorted list: it compares with the middle element and throws away half of the remaining list in every step. It therefore needs only about log2(n) comparisons.

3.5 Binary search

Growth rates matter more than fast hardware. For a million elements, an O(n^2) method performs around a trillion steps, while an O(n log n) method needs about twenty million. A better algorithm can turn a computation of days into one of seconds.

An algorithm is a finite list of precise steps that solves a problem for every allowed input. A recipe is a familiar example, but an algorithm must be exact enough that a machine can follow it without asking questions. Correctness and efficiency are the two properties we care about most.

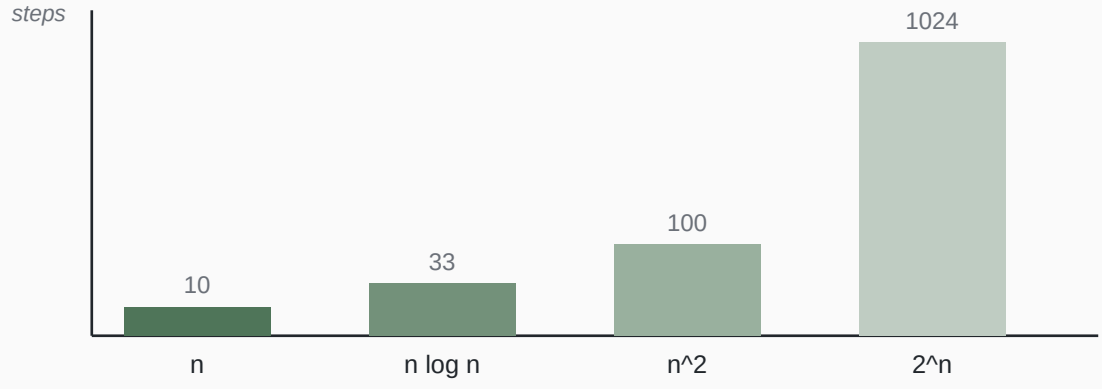


Figure 3.3 Steps for n = 10 (square-root scale)

To compare algorithms we count basic steps as a function of the input size n. We are mostly interested in how this count grows when n becomes large, so we ignore constant factors and write the result in big-O notation, such as O(n), O(n log n) or O(n^2).

Linear search looks at the elements one by one and needs up to n comparisons. Binary search works on a sorted list: it compares with the middle element and throws away half of the remaining list in every step. It therefore needs only about log2(n) comparisons.

Sorting is one of the most studied problems. Simple methods like insertion sort take O(n^2) steps in the worst case. Merge sort splits the list in halves, sorts each half and merges the results, which gives O(n log n) steps for every input.

3.6 Sorting

An algorithm is a finite list of precise steps that solves a problem for every allowed input. A recipe is a familiar example, but an algorithm must be exact enough that a machine can follow it without asking questions. Correctness and efficiency are the two properties we care about most.

To compare algorithms we count basic steps as a function of the input size n. We are mostly interested in how this count grows when n becomes large, so we ignore constant factors and write the result in big-O notation, such as O(n), O(n log n) or O(n^2).

> **Key idea**
> **For large inputs, the growth rate of an algorithm matters more than the speed of the machine.**

Linear search looks at the elements one by one and needs up to n comparisons. Binary search works on a sorted list: it compares with the middle element and throws away half of the remaining list in every step. It therefore needs only about log2(n) comparisons.

Sorting is one of the most studied problems. Simple methods like insertion sort take O(n^2) steps in the worst case. Merge sort splits the list in halves, sorts each half and merges the results, which gives O(n log n) steps for every input.

Growth rates matter more than fast hardware. For a million elements, an O(n^2) method performs around a trillion steps, while an O(n log n) method needs about twenty million. A better algorithm can turn a computation of days into one of seconds.

3.7 Merge sort

To compare algorithms we count basic steps as a function of the input size n. We are mostly interested in how this count grows when n becomes large, so we ignore constant factors and write the result in big-O notation, such as O(n), O(n log n) or O(n^2).

Linear search looks at the elements one by one and needs up to n comparisons. Binary search works on a sorted list: it compares with the middle element and throws away half of the remaining list in every step. It therefore needs only about log2(n) comparisons.

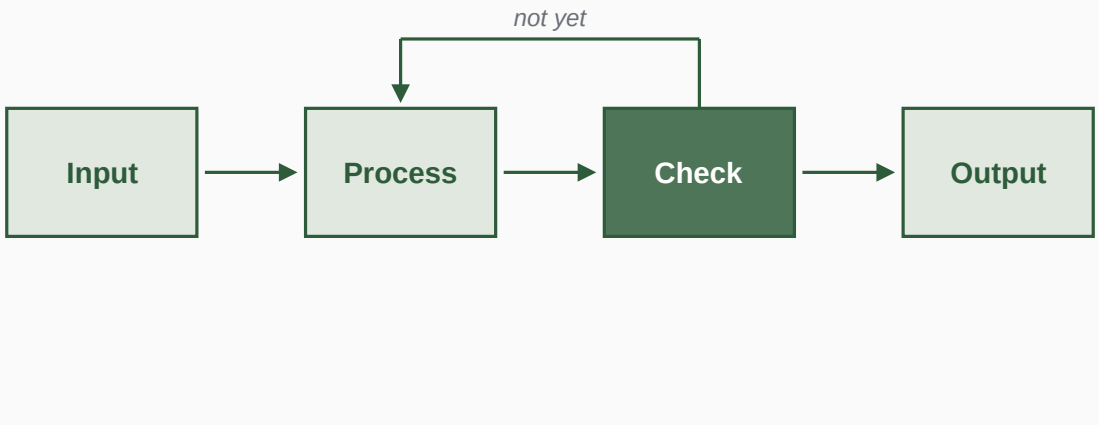


Figure 3.4 A process with a check that repeats a step

Sorting is one of the most studied problems. Simple methods like insertion sort take O(n^2) steps in the worst case. Merge sort splits the list in halves, sorts each half and merges the results, which gives O(n log n) steps for every input.

Growth rates matter more than fast hardware. For a million elements, an O(n^2) method performs around a trillion steps, while an O(n log n) method needs about twenty million. A better algorithm can turn a computation of days into one of seconds.

An algorithm is a finite list of precise steps that solves a problem for every allowed input. A recipe is a familiar example, but an algorithm must be exact enough that a machine can follow it without asking questions. Correctness and efficiency are the two properties we care about most.

3.8 Growth rates

Linear search looks at the elements one by one and needs up to n comparisons. Binary search works on a sorted list: it compares with the middle element and throws away half of the remaining list in every step. It therefore needs only about log2(n) comparisons.

Sorting is one of the most studied problems. Simple methods like insertion sort take O(n^2) steps in the worst case. Merge sort splits the list in halves, sorts each half and merges the results, which gives O(n log n) steps for every input.

> **Key idea**
> **For large inputs, the growth rate of an algorithm matters more than the speed of the machine.**

Growth rates matter more than fast hardware. For a million elements, an O(n^2) method performs around a trillion steps, while an O(n log n) method needs about twenty million. A better algorithm can turn a computation of days into one of seconds.

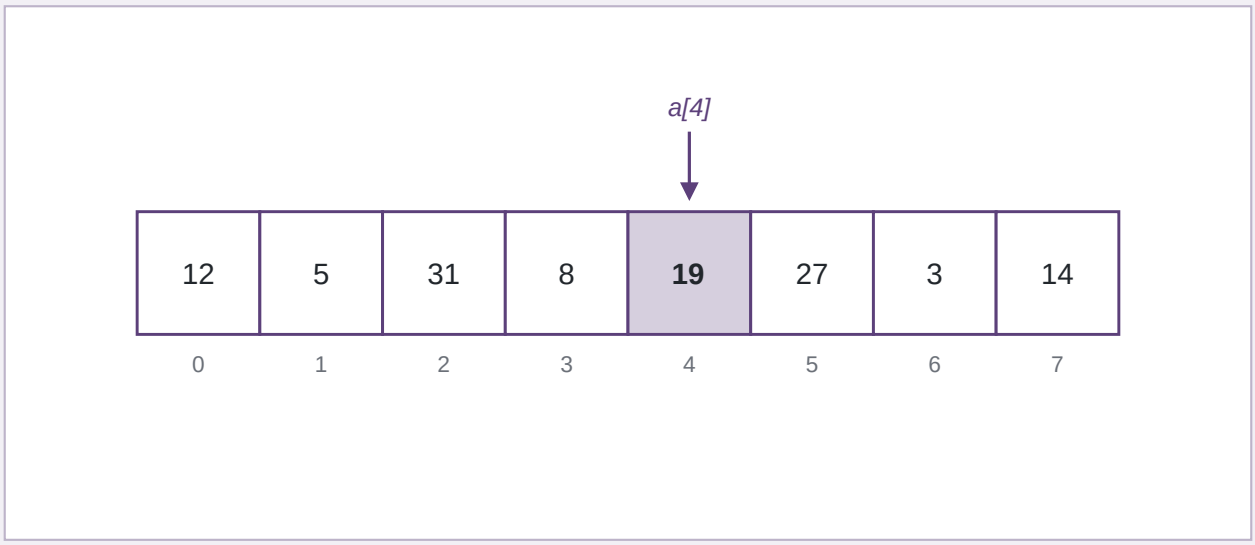
An algorithm is a finite list of precise steps that solves a problem for every allowed input. A recipe is a familiar example, but an algorithm must be exact enough that a machine can follow it without asking questions. Correctness and efficiency are the two properties we care about most.

To compare algorithms we count basic steps as a function of the input size n. We are mostly interested in how this count grows when n becomes large, so we ignore constant factors and write the result in big-O notation, such as O(n), O(n log n) or O(n^2).

Chapter 4

Data Structures

A data structure is a way to store data so that the operations we need are fast. The right choice depends on the questions we ask: do we look up items by position, by key, or do we always need the smallest one? There is no structure that is best for everything.



4.1 Choosing a structure

A data structure is a way to store data so that the operations we need are fast. The right choice depends on the questions we ask: do we look up items by position, by key, or do we always need the smallest one? There is no structure that is best for everything.

An array stores elements next to each other in memory. Reading the element at a given index takes constant time, but inserting in the middle requires moving all later elements. A linked list makes insertion cheap, but finding the k-th element means walking through k nodes.

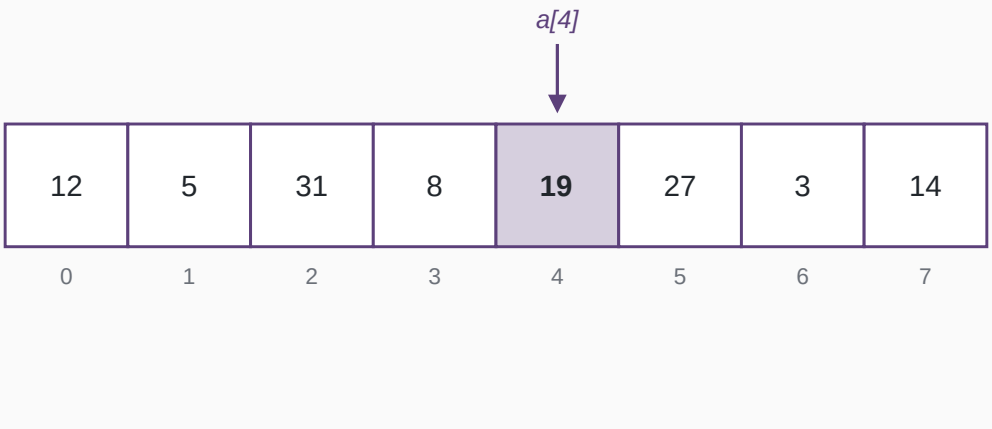


Figure 4.1 An array with its indices

Stacks and queues restrict how we access the data. A stack returns the most recently added element first (last in, first out), like a pile of plates. A queue returns the oldest element first (first in, first out), like people waiting at a counter.

A hash table maps keys to values. A hash function turns each key into an index of an array, so lookups take constant time on average. Two keys can land on the same index; good tables handle these collisions with short lists or by probing the next free slot.

Trees organise data hierarchically. In a binary search tree, every node is larger than all nodes in its left subtree and smaller than all nodes in its right subtree. If the tree stays balanced, searching, inserting and deleting all take O(log n) steps.

4.2 Arrays

An array stores elements next to each other in memory. Reading the element at a given index takes constant time, but inserting in the middle requires moving all later elements. A linked list makes insertion cheap, but finding the k-th element means walking through k nodes.

Stacks and queues restrict how we access the data. A stack returns the most recently added element first (last in, first out), like a pile of plates. A queue returns the oldest element first (first in, first out), like people waiting at a counter.

> **Key idea**
> **Pick the structure from the questions you ask most often, not from habit.**

A hash table maps keys to values. A hash function turns each key into an index of an array, so lookups take constant time on average. Two keys can land on the same index; good tables handle these collisions with short lists or by probing the next free slot.

Trees organise data hierarchically. In a binary search tree, every node is larger than all nodes in its left subtree and smaller than all nodes in its right subtree. If the tree stays balanced, searching, inserting and deleting all take O(log n) steps.

A data structure is a way to store data so that the operations we need are fast. The right choice depends on the questions we ask: do we look up items by position, by key, or do we always need the smallest one? There is no structure that is best for everything.

4.3 Linked lists

Stacks and queues restrict how we access the data. A stack returns the most recently added element first (last in, first out), like a pile of plates. A queue returns the oldest element first (first in, first out), like people waiting at a counter.

A hash table maps keys to values. A hash function turns each key into an index of an array, so lookups take constant time on average. Two keys can land on the same index; good tables handle these collisions with short lists or by probing the next free slot.

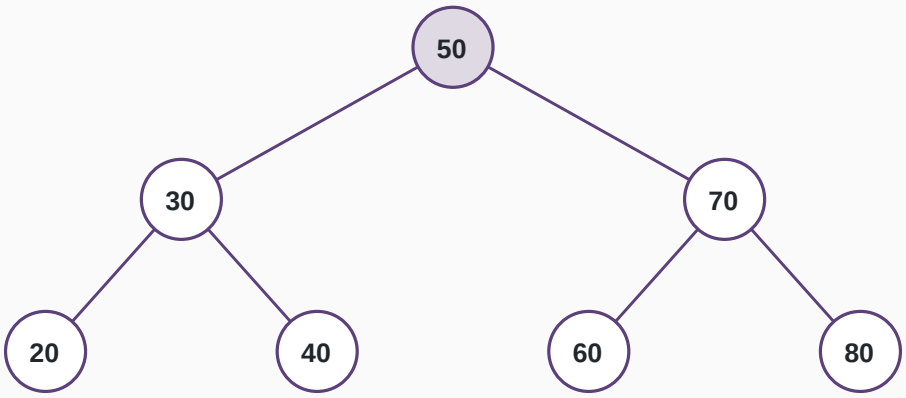


Figure 4.2 A binary search tree

Trees organise data hierarchically. In a binary search tree, every node is larger than all nodes in its left subtree and smaller than all nodes in its right subtree. If the tree stays balanced, searching, inserting and deleting all take O(log n) steps.

A data structure is a way to store data so that the operations we need are fast. The right choice depends on the questions we ask: do we look up items by position, by key, or do we always need the smallest one? There is no structure that is best for everything.

An array stores elements next to each other in memory. Reading the element at a given index takes constant time, but inserting in the middle requires moving all later elements. A linked list makes insertion cheap, but finding the k-th element means walking through k nodes.

4.4 Stacks

A hash table maps keys to values. A hash function turns each key into an index of an array, so lookups take constant time on average. Two keys can land on the same index; good tables handle these collisions with short lists or by probing the next free slot.

Trees organise data hierarchically. In a binary search tree, every node is larger than all nodes in its left subtree and smaller than all nodes in its right subtree. If the tree stays balanced, searching, inserting and deleting all take O(log n) steps.

> **Key idea**
> **Pick the structure from the questions you ask most often, not from habit.**

A data structure is a way to store data so that the operations we need are fast. The right choice depends on the questions we ask: do we look up items by position, by key, or do we always need the smallest one? There is no structure that is best for everything.

An array stores elements next to each other in memory. Reading the element at a given index takes constant time, but inserting in the middle requires moving all later elements. A linked list makes insertion cheap, but finding the k-th element means walking through k nodes.

Stacks and queues restrict how we access the data. A stack returns the most recently added element first (last in, first out), like a pile of plates. A queue returns the oldest element first (first in, first out), like people waiting at a counter.

4.5 Queues

Trees organise data hierarchically. In a binary search tree, every node is larger than all nodes in its left subtree and smaller than all nodes in its right subtree. If the tree stays balanced, searching, inserting and deleting all take O(log n) steps.

A data structure is a way to store data so that the operations we need are fast. The right choice depends on the questions we ask: do we look up items by position, by key, or do we always need the smallest one? There is no structure that is best for everything.

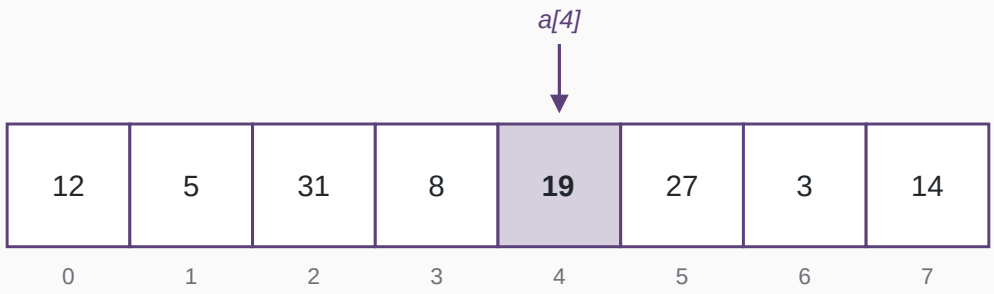


Figure 4.3 An array with its indices

An array stores elements next to each other in memory. Reading the element at a given index takes constant time, but inserting in the middle requires moving all later elements. A linked list makes insertion cheap, but finding the k-th element means walking through k nodes.

Stacks and queues restrict how we access the data. A stack returns the most recently added element first (last in, first out), like a pile of plates. A queue returns the oldest element first (first in, first out), like people waiting at a counter.

A hash table maps keys to values. A hash function turns each key into an index of an array, so lookups take constant time on average. Two keys can land on the same index; good tables handle these collisions with short lists or by probing the next free slot.

4.6 Hash tables

A data structure is a way to store data so that the operations we need are fast. The right choice depends on the questions we ask: do we look up items by position, by key, or do we always need the smallest one? There is no structure that is best for everything.

An array stores elements next to each other in memory. Reading the element at a given index takes constant time, but inserting in the middle requires moving all later elements. A linked list makes insertion cheap, but finding the k-th element means walking through k nodes.

> **Key idea**
> **Pick the structure from the questions you ask most often, not from habit.**

Stacks and queues restrict how we access the data. A stack returns the most recently added element first (last in, first out), like a pile of plates. A queue returns the oldest element first (first in, first out), like people waiting at a counter.

A hash table maps keys to values. A hash function turns each key into an index of an array, so lookups take constant time on average. Two keys can land on the same index; good tables handle these collisions with short lists or by probing the next free slot.

Trees organise data hierarchically. In a binary search tree, every node is larger than all nodes in its left subtree and smaller than all nodes in its right subtree. If the tree stays balanced, searching, inserting and deleting all take O(log n) steps.

4.7 Trees

An array stores elements next to each other in memory. Reading the element at a given index takes constant time, but inserting in the middle requires moving all later elements. A linked list makes insertion cheap, but finding the k-th element means walking through k nodes.

Stacks and queues restrict how we access the data. A stack returns the most recently added element first (last in, first out), like a pile of plates. A queue returns the oldest element first (first in, first out), like people waiting at a counter.

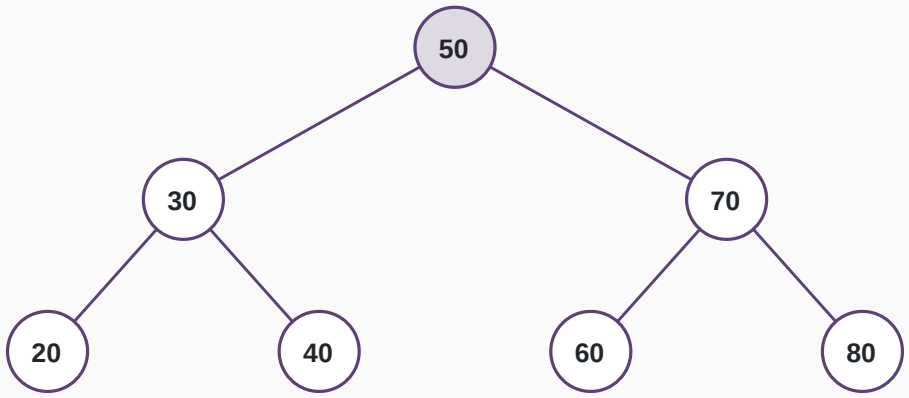


Figure 4.4 A binary search tree

A hash table maps keys to values. A hash function turns each key into an index of an array, so lookups take constant time on average. Two keys can land on the same index; good tables handle these collisions with short lists or by probing the next free slot.

Trees organise data hierarchically. In a binary search tree, every node is larger than all nodes in its left subtree and smaller than all nodes in its right subtree. If the tree stays balanced, searching, inserting and deleting all take O(log n) steps.

A data structure is a way to store data so that the operations we need are fast. The right choice depends on the questions we ask: do we look up items by position, by key, or do we always need the smallest one? There is no structure that is best for everything.

4.8 Balanced trees

Stacks and queues restrict how we access the data. A stack returns the most recently added element first (last in, first out), like a pile of plates. A queue returns the oldest element first (first in, first out), like people waiting at a counter.

A hash table maps keys to values. A hash function turns each key into an index of an array, so lookups take constant time on average. Two keys can land on the same index; good tables handle these collisions with short lists or by probing the next free slot.

> **Key idea**
> **Pick the structure from the questions you ask most often, not from habit.**

Trees organise data hierarchically. In a binary search tree, every node is larger than all nodes in its left subtree and smaller than all nodes in its right subtree. If the tree stays balanced, searching, inserting and deleting all take O(log n) steps.

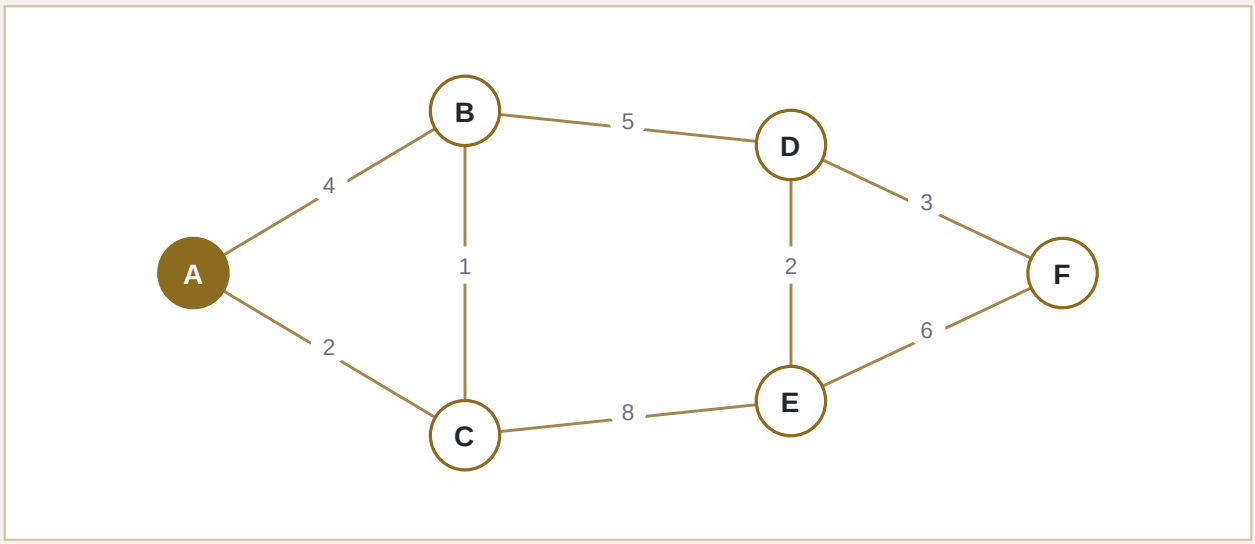
A data structure is a way to store data so that the operations we need are fast. The right choice depends on the questions we ask: do we look up items by position, by key, or do we always need the smallest one? There is no structure that is best for everything.

An array stores elements next to each other in memory. Reading the element at a given index takes constant time, but inserting in the middle requires moving all later elements. A linked list makes insertion cheap, but finding the k-th element means walking through k nodes.

Chapter 5

Graphs

A graph consists of vertices and edges that connect pairs of vertices. Road maps, social networks, the links between web pages and the dependencies between course modules are all graphs. Many different questions become the same question once we see the graph behind them.



5.1 Vertices and edges

A graph consists of vertices and edges that connect pairs of vertices. Road maps, social networks, the links between web pages and the dependencies between course modules are all graphs. Many different questions become the same question once we see the graph behind them.

Edges can be directed, like one-way streets, or undirected. They can also carry weights, such as distances or costs. A path is a sequence of edges that leads from one vertex to another, and a cycle is a path that returns to its starting point.

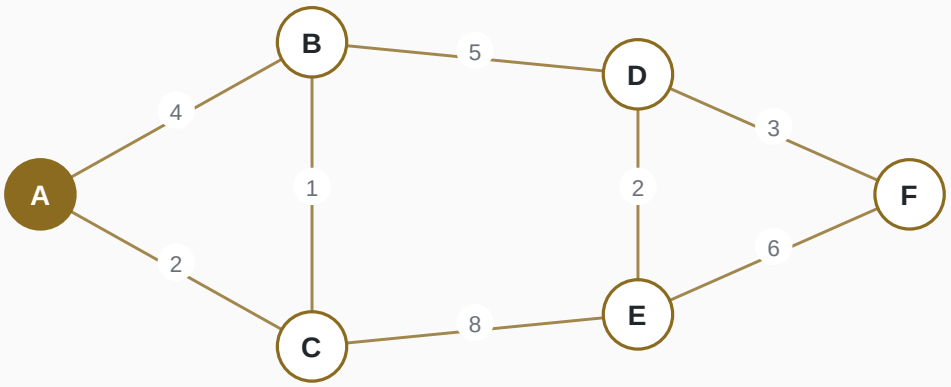


Figure 5.1 A weighted graph with six vertices

Breadth-first search explores a graph layer by layer: first all neighbours of the start vertex, then their neighbours, and so on. In a graph without weights it finds shortest paths. Depth-first search instead follows one path as far as possible before it backtracks.

Dijkstra's algorithm finds shortest paths when edges have non-negative weights. It keeps a set of vertices whose distance is final and repeatedly adds the closest remaining vertex. With a priority queue it runs in O((V + E) log V) time.

A tree is a connected graph without cycles. A spanning tree connects all vertices of a graph using as few edges as possible. Choosing the cheapest such tree is the minimum spanning tree problem, which simple greedy algorithms solve exactly.

5.2 Directed and weighted graphs

Edges can be directed, like one-way streets, or undirected. They can also carry weights, such as distances or costs. A path is a sequence of edges that leads from one vertex to another, and a cycle is a path that returns to its starting point.

Breadth-first search explores a graph layer by layer: first all neighbours of the start vertex, then their neighbours, and so on. In a graph without weights it finds shortest paths. Depth-first search instead follows one path as far as possible before it backtracks.

> **Key idea**
> **Many different problems become the same problem once you see the graph behind them.**

Dijkstra's algorithm finds shortest paths when edges have non-negative weights. It keeps a set of vertices whose distance is final and repeatedly adds the closest remaining vertex. With a priority queue it runs in O((V + E) log V) time.

A tree is a connected graph without cycles. A spanning tree connects all vertices of a graph using as few edges as possible. Choosing the cheapest such tree is the minimum spanning tree problem, which simple greedy algorithms solve exactly.

A graph consists of vertices and edges that connect pairs of vertices. Road maps, social networks, the links between web pages and the dependencies between course modules are all graphs. Many different questions become the same question once we see the graph behind them.

5.3 Paths and cycles

Breadth-first search explores a graph layer by layer: first all neighbours of the start vertex, then their neighbours, and so on. In a graph without weights it finds shortest paths. Depth-first search instead follows one path as far as possible before it backtracks.

Dijkstra's algorithm finds shortest paths when edges have non-negative weights. It keeps a set of vertices whose distance is final and repeatedly adds the closest remaining vertex. With a priority queue it runs in O((V + E) log V) time.

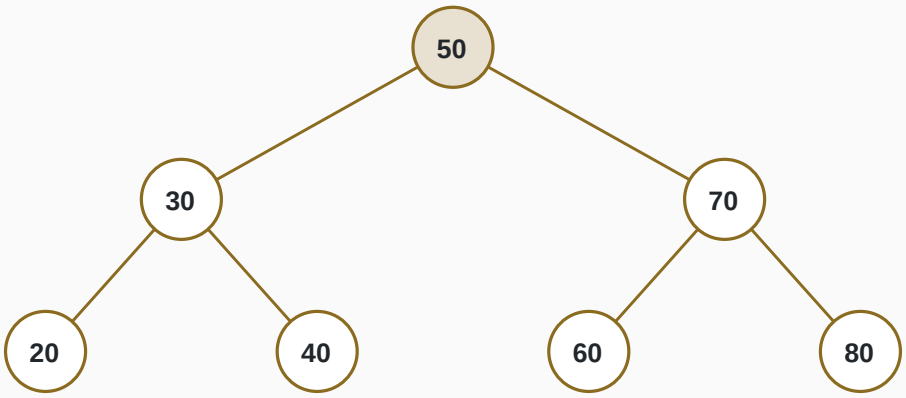


Figure 5.2 A binary search tree

A tree is a connected graph without cycles. A spanning tree connects all vertices of a graph using as few edges as possible. Choosing the cheapest such tree is the minimum spanning tree problem, which simple greedy algorithms solve exactly.

A graph consists of vertices and edges that connect pairs of vertices. Road maps, social networks, the links between web pages and the dependencies between course modules are all graphs. Many different questions become the same question once we see the graph behind them.

Edges can be directed, like one-way streets, or undirected. They can also carry weights, such as distances or costs. A path is a sequence of edges that leads from one vertex to another, and a cycle is a path that returns to its starting point.

5.4 Breadth-first search

Dijkstra's algorithm finds shortest paths when edges have non-negative weights. It keeps a set of vertices whose distance is final and repeatedly adds the closest remaining vertex. With a priority queue it runs in O((V + E) log V) time.

A tree is a connected graph without cycles. A spanning tree connects all vertices of a graph using as few edges as possible. Choosing the cheapest such tree is the minimum spanning tree problem, which simple greedy algorithms solve exactly.

> **Key idea**
> **Many different problems become the same problem once you see the graph behind them.**

A graph consists of vertices and edges that connect pairs of vertices. Road maps, social networks, the links between web pages and the dependencies between course modules are all graphs. Many different questions become the same question once we see the graph behind them.

Edges can be directed, like one-way streets, or undirected. They can also carry weights, such as distances or costs. A path is a sequence of edges that leads from one vertex to another, and a cycle is a path that returns to its starting point.

Breadth-first search explores a graph layer by layer: first all neighbours of the start vertex, then their neighbours, and so on. In a graph without weights it finds shortest paths. Depth-first search instead follows one path as far as possible before it backtracks.

5.5 Depth-first search

A tree is a connected graph without cycles. A spanning tree connects all vertices of a graph using as few edges as possible. Choosing the cheapest such tree is the minimum spanning tree problem, which simple greedy algorithms solve exactly.

A graph consists of vertices and edges that connect pairs of vertices. Road maps, social networks, the links between web pages and the dependencies between course modules are all graphs. Many different questions become the same question once we see the graph behind them.

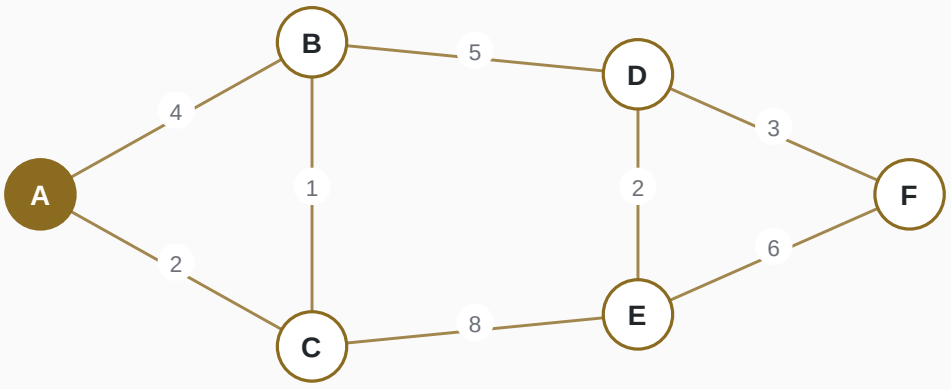


Figure 5.3 A weighted graph with six vertices

Edges can be directed, like one-way streets, or undirected. They can also carry weights, such as distances or costs. A path is a sequence of edges that leads from one vertex to another, and a cycle is a path that returns to its starting point.

Breadth-first search explores a graph layer by layer: first all neighbours of the start vertex, then their neighbours, and so on. In a graph without weights it finds shortest paths. Depth-first search instead follows one path as far as possible before it backtracks.

Dijkstra's algorithm finds shortest paths when edges have non-negative weights. It keeps a set of vertices whose distance is final and repeatedly adds the closest remaining vertex. With a priority queue it runs in O((V + E) log V) time.

5.6 Shortest paths

A graph consists of vertices and edges that connect pairs of vertices. Road maps, social networks, the links between web pages and the dependencies between course modules are all graphs. Many different questions become the same question once we see the graph behind them.

Edges can be directed, like one-way streets, or undirected. They can also carry weights, such as distances or costs. A path is a sequence of edges that leads from one vertex to another, and a cycle is a path that returns to its starting point.

> **Key idea**
> **Many different problems become the same problem once you see the graph behind them.**

Breadth-first search explores a graph layer by layer: first all neighbours of the start vertex, then their neighbours, and so on. In a graph without weights it finds shortest paths. Depth-first search instead follows one path as far as possible before it backtracks.

Dijkstra's algorithm finds shortest paths when edges have non-negative weights. It keeps a set of vertices whose distance is final and repeatedly adds the closest remaining vertex. With a priority queue it runs in O((V + E) log V) time.

A tree is a connected graph without cycles. A spanning tree connects all vertices of a graph using as few edges as possible. Choosing the cheapest such tree is the minimum spanning tree problem, which simple greedy algorithms solve exactly.

5.7 Trees in graphs

Edges can be directed, like one-way streets, or undirected. They can also carry weights, such as distances or costs. A path is a sequence of edges that leads from one vertex to another, and a cycle is a path that returns to its starting point.

Breadth-first search explores a graph layer by layer: first all neighbours of the start vertex, then their neighbours, and so on. In a graph without weights it finds shortest paths. Depth-first search instead follows one path as far as possible before it backtracks.

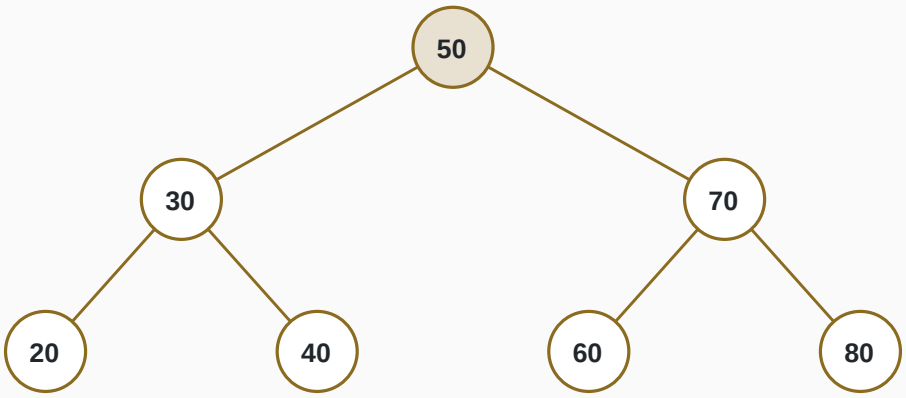


Figure 5.4 A binary search tree

Dijkstra's algorithm finds shortest paths when edges have non-negative weights. It keeps a set of vertices whose distance is final and repeatedly adds the closest remaining vertex. With a priority queue it runs in O((V + E) log V) time.

A tree is a connected graph without cycles. A spanning tree connects all vertices of a graph using as few edges as possible. Choosing the cheapest such tree is the minimum spanning tree problem, which simple greedy algorithms solve exactly.

A graph consists of vertices and edges that connect pairs of vertices. Road maps, social networks, the links between web pages and the dependencies between course modules are all graphs. Many different questions become the same question once we see the graph behind them.

5.8 Spanning trees

Breadth-first search explores a graph layer by layer: first all neighbours of the start vertex, then their neighbours, and so on. In a graph without weights it finds shortest paths. Depth-first search instead follows one path as far as possible before it backtracks.

Dijkstra's algorithm finds shortest paths when edges have non-negative weights. It keeps a set of vertices whose distance is final and repeatedly adds the closest remaining vertex. With a priority queue it runs in O((V + E) log V) time.

> **Key idea**
> **Many different problems become the same problem once you see the graph behind them.**

A tree is a connected graph without cycles. A spanning tree connects all vertices of a graph using as few edges as possible. Choosing the cheapest such tree is the minimum spanning tree problem, which simple greedy algorithms solve exactly.

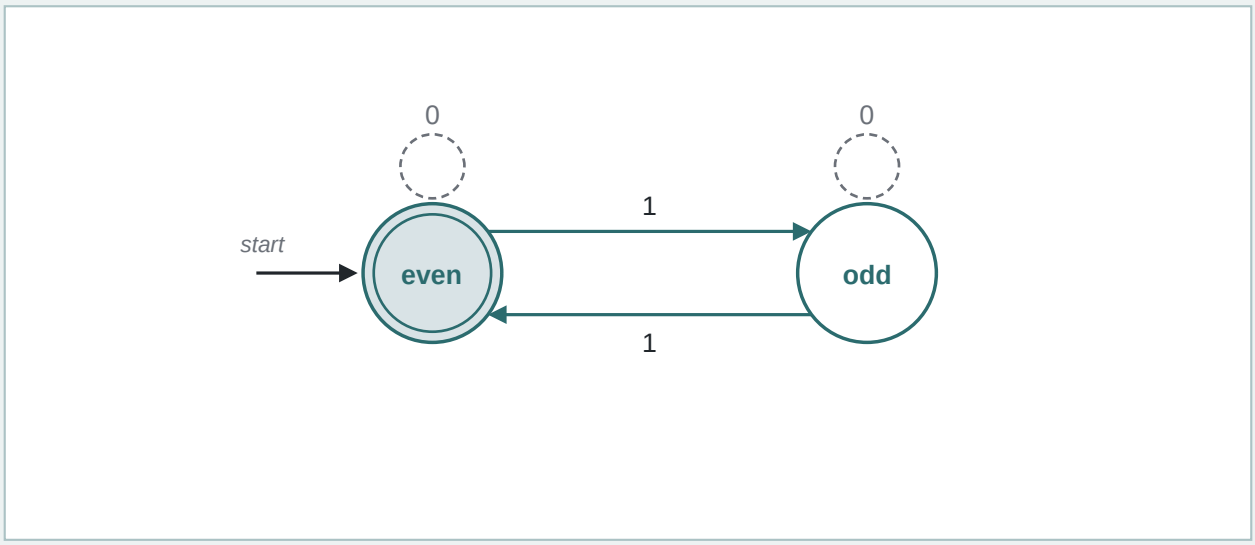
A graph consists of vertices and edges that connect pairs of vertices. Road maps, social networks, the links between web pages and the dependencies between course modules are all graphs. Many different questions become the same question once we see the graph behind them.

Edges can be directed, like one-way streets, or undirected. They can also carry weights, such as distances or costs. A path is a sequence of edges that leads from one vertex to another, and a cycle is a path that returns to its starting point.

Chapter 6

Finite Automata

A finite automaton is a very small model of a computer. It has a finite set of states, a start state, some accepting states and a transition rule that tells it where to go after reading each input symbol. It reads a word from left to right and accepts or rejects it.



6.1 States and transitions

A finite automaton is a very small model of a computer. It has a finite set of states, a start state, some accepting states and a transition rule that tells it where to go after reading each input symbol. It reads a word from left to right and accepts or rejects it.

The set of all words an automaton accepts is its language. For example, an automaton with two states can decide whether a binary word contains an even number of ones: it switches between 'even' and 'odd' on every one and stays put on every zero.

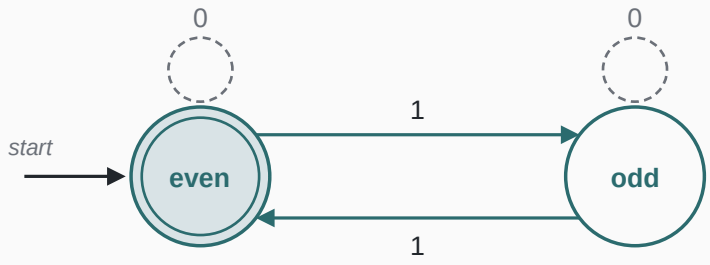


Figure 6.1 Automaton for an even number of ones

Regular expressions describe the same languages as finite automata. The expression (ab)* matches the empty word, ab, abab and so on. Search tools and input validators translate regular expressions into automata behind the scenes.

Not every language is regular. An automaton cannot count without limit, so it cannot check whether a word has the form a^n b^n with equally many a's and b's. Proving such limits is done with the pumping lemma.

Stronger models, such as pushdown automata and Turing machines, add memory to the finite control. The Turing machine is simple enough to analyse and yet strong enough to compute everything that any real computer can compute.

6.2 Accepting words

The set of all words an automaton accepts is its language. For example, an automaton with two states can decide whether a binary word contains an even number of ones: it switches between 'even' and 'odd' on every one and stays put on every zero.

Regular expressions describe the same languages as finite automata. The expression (ab)* matches the empty word, ab, abab and so on. Search tools and input validators translate regular expressions into automata behind the scenes.

> **Key idea**
> **A finite automaton remembers only its current state, so it cannot count without limit.**

Not every language is regular. An automaton cannot count without limit, so it cannot check whether a word has the form a^n b^n with equally many a's and b's. Proving such limits is done with the pumping lemma.

Stronger models, such as pushdown automata and Turing machines, add memory to the finite control. The Turing machine is simple enough to analyse and yet strong enough to compute everything that any real computer can compute.

A finite automaton is a very small model of a computer. It has a finite set of states, a start state, some accepting states and a transition rule that tells it where to go after reading each input symbol. It reads a word from left to right and accepts or rejects it.

6.3 Languages

Regular expressions describe the same languages as finite automata. The expression (ab)* matches the empty word, ab, abab and so on. Search tools and input validators translate regular expressions into automata behind the scenes.

Not every language is regular. An automaton cannot count without limit, so it cannot check whether a word has the form a^n b^n with equally many a's and b's. Proving such limits is done with the pumping lemma.

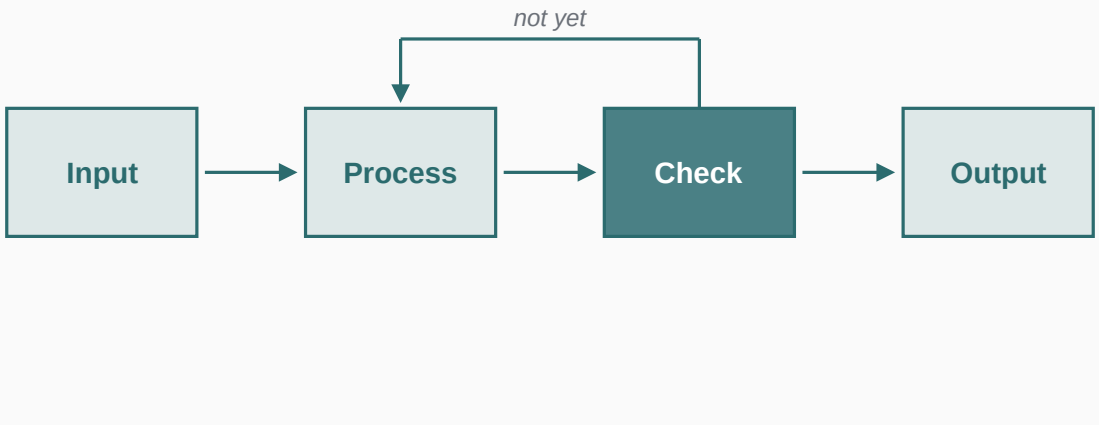


Figure 6.2 A process with a check that repeats a step

Stronger models, such as pushdown automata and Turing machines, add memory to the finite control. The Turing machine is simple enough to analyse and yet strong enough to compute everything that any real computer can compute.

A finite automaton is a very small model of a computer. It has a finite set of states, a start state, some accepting states and a transition rule that tells it where to go after reading each input symbol. It reads a word from left to right and accepts or rejects it.

The set of all words an automaton accepts is its language. For example, an automaton with two states can decide whether a binary word contains an even number of ones: it switches between 'even' and 'odd' on every one and stays put on every zero.

6.4 An example automaton

Not every language is regular. An automaton cannot count without limit, so it cannot check whether a word has the form a^n b^n with equally many a's and b's. Proving such limits is done with the pumping lemma.

Stronger models, such as pushdown automata and Turing machines, add memory to the finite control. The Turing machine is simple enough to analyse and yet strong enough to compute everything that any real computer can compute.

> **Key idea**
> **A finite automaton remembers only its current state, so it cannot count without limit.**

A finite automaton is a very small model of a computer. It has a finite set of states, a start state, some accepting states and a transition rule that tells it where to go after reading each input symbol. It reads a word from left to right and accepts or rejects it.

The set of all words an automaton accepts is its language. For example, an automaton with two states can decide whether a binary word contains an even number of ones: it switches between 'even' and 'odd' on every one and stays put on every zero.

Regular expressions describe the same languages as finite automata. The expression (ab)* matches the empty word, ab, abab and so on. Search tools and input validators translate regular expressions into automata behind the scenes.

6.5 Regular expressions

Stronger models, such as pushdown automata and Turing machines, add memory to the finite control. The Turing machine is simple enough to analyse and yet strong enough to compute everything that any real computer can compute.

A finite automaton is a very small model of a computer. It has a finite set of states, a start state, some accepting states and a transition rule that tells it where to go after reading each input symbol. It reads a word from left to right and accepts or rejects it.

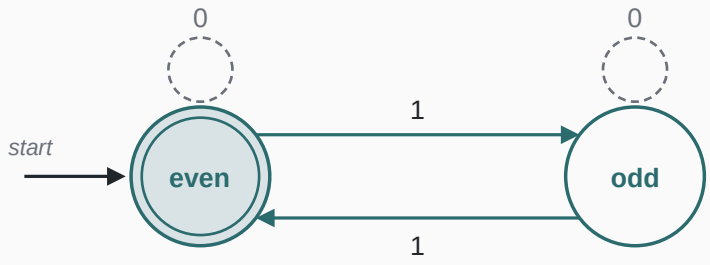


Figure 6.3 Automaton for an even number of ones

The set of all words an automaton accepts is its language. For example, an automaton with two states can decide whether a binary word contains an even number of ones: it switches between 'even' and 'odd' on every one and stays put on every zero.

Regular expressions describe the same languages as finite automata. The expression (ab)* matches the empty word, ab, abab and so on. Search tools and input validators translate regular expressions into automata behind the scenes.

Not every language is regular. An automaton cannot count without limit, so it cannot check whether a word has the form a^n b^n with equally many a's and b's. Proving such limits is done with the pumping lemma.

6.6 Limits of automata

A finite automaton is a very small model of a computer. It has a finite set of states, a start state, some accepting states and a transition rule that tells it where to go after reading each input symbol. It reads a word from left to right and accepts or rejects it.

The set of all words an automaton accepts is its language. For example, an automaton with two states can decide whether a binary word contains an even number of ones: it switches between 'even' and 'odd' on every one and stays put on every zero.

> **Key idea**
> **A finite automaton remembers only its current state, so it cannot count without limit.**

Regular expressions describe the same languages as finite automata. The expression (ab)* matches the empty word, ab, abab and so on. Search tools and input validators translate regular expressions into automata behind the scenes.

Not every language is regular. An automaton cannot count without limit, so it cannot check whether a word has the form a^n b^n with equally many a's and b's. Proving such limits is done with the pumping lemma.

Stronger models, such as pushdown automata and Turing machines, add memory to the finite control. The Turing machine is simple enough to analyse and yet strong enough to compute everything that any real computer can compute.

6.7 Stronger models

The set of all words an automaton accepts is its language. For example, an automaton with two states can decide whether a binary word contains an even number of ones: it switches between 'even' and 'odd' on every one and stays put on every zero.

Regular expressions describe the same languages as finite automata. The expression (ab)* matches the empty word, ab, abab and so on. Search tools and input validators translate regular expressions into automata behind the scenes.

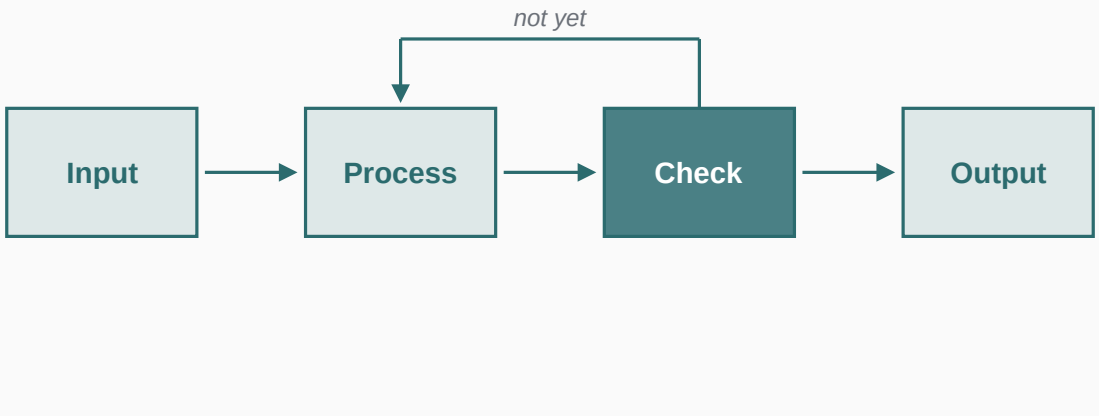


Figure 6.4 A process with a check that repeats a step

Not every language is regular. An automaton cannot count without limit, so it cannot check whether a word has the form a^n b^n with equally many a's and b's. Proving such limits is done with the pumping lemma.

Stronger models, such as pushdown automata and Turing machines, add memory to the finite control. The Turing machine is simple enough to analyse and yet strong enough to compute everything that any real computer can compute.

A finite automaton is a very small model of a computer. It has a finite set of states, a start state, some accepting states and a transition rule that tells it where to go after reading each input symbol. It reads a word from left to right and accepts or rejects it.

6.8 Summary

Regular expressions describe the same languages as finite automata. The expression (ab)* matches the empty word, ab, abab and so on. Search tools and input validators translate regular expressions into automata behind the scenes.

Not every language is regular. An automaton cannot count without limit, so it cannot check whether a word has the form a^n b^n with equally many a's and b's. Proving such limits is done with the pumping lemma.

> **Key idea**
> **A finite automaton remembers only its current state, so it cannot count without limit.**

Stronger models, such as pushdown automata and Turing machines, add memory to the finite control. The Turing machine is simple enough to analyse and yet strong enough to compute everything that any real computer can compute.

A finite automaton is a very small model of a computer. It has a finite set of states, a start state, some accepting states and a transition rule that tells it where to go after reading each input symbol. It reads a word from left to right and accepts or rejects it.

The set of all words an automaton accepts is its language. For example, an automaton with two states can decide whether a binary word contains an even number of ones: it switches between 'even' and 'odd' on every one and stays put on every zero.

Exercises

1. Write the truth table of (p and not q) or (not p and q). Which connective from the chapter does it describe?

2. Use De Morgan's laws to simplify the condition not (x > 0 and y > 0) in a program.

3. Let A = {1, 2, 3, 4} and B = {3, 4, 5}. Write down the union, the intersection and A minus B.

Exercises (continued)

4. Give a function from {1, 2, 3} to {a, b} that is surjective. Can such a function be injective? Explain.

5. How many comparisons does binary search need at most for a sorted list with 1000 elements?

6. Sort the list 7, 3, 9, 1, 5, 2 with merge sort and write down every merge step.

Exercises (continued)

7. Which data structure would you use for the undo history of a text editor? Give a reason.

8. Insert 50, 30, 70, 20, 40, 60, 80 into an empty binary search tree. Draw the result.

9. Draw a graph with five vertices that has a cycle of length four. Find a spanning tree of it.

Exercises (continued)

10. Run breadth-first search on your graph from exercise 9 and list the vertices in the order they are visited.

11. Draw an automaton that accepts exactly the binary words that end with 01.

12. Write a regular expression for all words over {a, b} that contain the letter a at least twice.

Glossary

Algorithm	A finite list of precise steps that solves a problem for every allowed input.
Automaton	A model with finitely many states that reads a word and accepts or rejects it.
Big-O	Notation for the growth of a step count, ignoring constant factors.
Bijection	A function that is both injective and surjective; it can be reversed.
Binary search	Search in a sorted list that halves the remaining part in every step.
Breadth-first search	Graph search that visits vertices layer by layer from the start.
Contradiction	A formula that is false for every assignment of its variables.
Cycle	A path in a graph that returns to its starting vertex.
Domain	The set of inputs of a function.
Graph	A set of vertices together with edges between pairs of vertices.
Hash table	A structure that maps keys to array positions with a hash function.

Glossary

Implication	The connective 'if p then q'; false only when p is true and q is false.
Language	The set of all words that an automaton accepts.
Merge sort	Sorting by splitting in halves, sorting them and merging the results.
Proposition	A sentence that is either true or false.
Queue	A structure that returns the oldest element first (first in, first out).
Relation	Any set of pairs (a, b) between two sets.
Spanning tree	A tree inside a graph that connects all of its vertices.
Stack	A structure that returns the newest element first (last in, first out).
Tautology	A formula that is true for every assignment of its variables.
Tree	A connected graph without cycles.
Truth table	A table that lists the value of a formula for every assignment.

End of the demo document

64 pages, generated by scripts/demo-pdf-uret.mjs